

Comparison of Optimizers in a Regression Model

Before diving into the main part of the tutorial, let's first establish some foundational concepts.

1 A brief Introduction to Neural Networks

A neural network is a machine learning system that organizes simple computational units (neurons) into layers, learning patterns in data by adjusting weights and biases to map inputs to corresponding outputs. (IBM, 2021).

The idea of neural networks stems from the biological neuron within the brain. It started off in 1943 by Warren McCulloch and Walter Pitts in their paper 'A logical calculus of the immanent in nervous activity' where they proposed the first mathematical model of a neuron, showing that simple units could perform computation of a function. This was then followed by Frank Rosenblatt in 1958 who introduced the perceptron, the foundation of all neural networks.

2 The Perceptron

A perceptron is modelled after a single neuron. Perceptron's are single-layer neural networks for binary classification. It performs classification by learning a decision boundary that separates the two classes (Medium, 2024). Figure 1 demonstrates the structure of a perceptron.

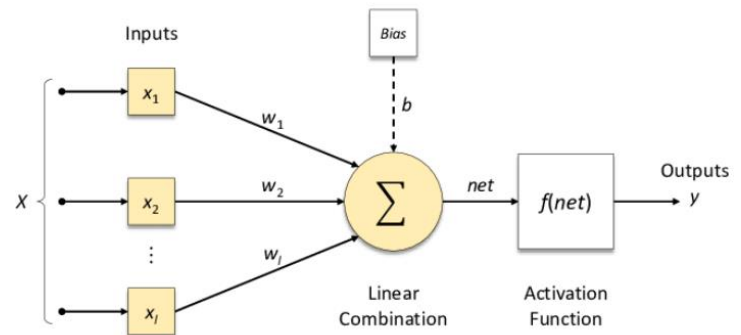


Figure 1 - The Structure of a Perceptron (Medium, 2024)

The structure of a perceptron is as follows:

- Inputs ($x_1, x_2, \dots, x_n$): These are the input data
- Weights ($w_1, w_2, \dots, w_n$): These are the parameters that determine the importance of each input
- Bias (b): Additional parameter allowing the decision boundary to be shifted
- Activation function: In a perceptron this is a step function producing binary outputs (1 or 0)
- Output (y): The result of the perceptron computation

2.1 Operation of a simple Perceptron

1. The perceptron first computes the weighted sum of its incoming signals by multiplying each input to its corresponding weight.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n = \sum_{i=1}^n w_ix_i$$

Equation 1 - Weighted Sum Equation

2. If the net output of the weighted sum exceeds some predetermined threshold value $\emptyset$, then the perceptron fires (its output is 1), otherwise it doesn't fire (its output is 0).

$$z = \sum_{i=1}^n w_ix_i \geq \emptyset$$

Equation 2 - Output of a Perceptron based on a specified threshold

2.2 Multi-Layered Simple Perceptrons

The weighted sum formula shown in Equation 1 is a linear transformation. Even if perceptron were to be stacked together as shown in Figure 2. Mathematically, it can only produce straight line classifiers or in higher dimensions, a hyperplane classifier. Even in a multi-layer perceptron were to be stacked, the resulting equation would still remain a linear transformation (CodeSignal, 2025).

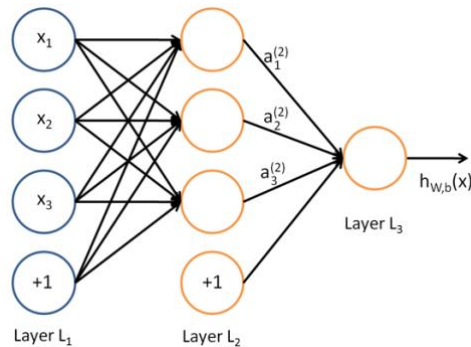


Figure 2 - Layered Simple Perceptrons (*deeplearning.stanford.edu, 2025*)

2.3 Building Non-Linearity in Multi-Layered Neural Networks through Activation Functions

In section 2, the perceptron model originally used a step function as its activation, producing a binary output when the weighted sum crossed a threshold. Replacing it with a non-linear activation, such as a sigmoid, transforms the neuron's output. While a single neuron with a non-linear activation still produces a linear decision boundary, stacking multiple neurons with non-linear activations in a multi-layer network enables the model to learn complex, non-linear patterns (CodeSignal, 2025).

If a feed forward network is fully connected (All outputs of the nodes of one layer is connected as inputs to all nodes of the next layer) and has a non-linear activation function, then it is called a Multi-Layer Perceptron (MLP) (ScienceDirect, 2023).

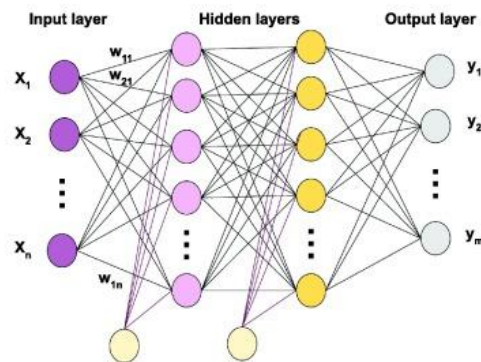


Figure 3 - A Multi-layered Perceptron (*ScienceDirect, 2023*)

3 Loss/Error function

Loss functions are a fundamental component of machine learning models because they measure how well a model's predictions match the true outcomes. A loss function quantifies the difference between the predicted values generated by the model and the actual values in the dataset. During training, the model attempts to minimize this difference, allowing it to improve its predictions and better capture the underlying patterns in

the data. There are several types of loss functions used for regression problems; however, in this tutorial we will focus on the Mean Squared Error (MSE) loss function (geeksforgeeks, 2025).

3.1 Mean Square Error

The Mean Squared Error (MSE) is a common loss function in machine learning where the mean of the squared residuals is taken rather than just the sum.

$$E = \frac{1}{n} \sum (y_i - \bar{y}_i)^2$$

Equation 3 -Mean Square Error formula (geeksforgeeks, 2025)

- y_i – Target value of index p
- $\bar{y}_i$ – Predicted value of index p
- n – Total number of datapoints

In Equation 3, residuals are the difference between the actual and the predicted prediction by the model. Squaring also has the added benefit of assigning more weight to larger errors.

3.2 Optimization

Optimization is a vital process when training a neural network model. Optimization occurs by finding the optimal set of parameters (weights and biases) that are able to best predict the data we want. The most common optimization technique is gradient descent (towardsdatascience, 2021).

$$\text{Gradient} = \frac{\delta E}{\delta W}$$

Equation 4 - Gradient Formula

- δE – change in error function(MSE)
- δW – change in weights

The gradient tells us how the error changes when the weights change. Then we update the weights based on these set of changes, this is known as gradient descent.

$$W_{\text{new}} \rightarrow W_{\text{old}} - \eta \frac{\delta E}{\delta W}$$

Equation 5 - Gradient Descent

- η – the ‘learning rate’ at which we update the weights

Then for every iteration, we continue this gradient descent until we reach convergence.

4 Optimizers

Gradient descent is the foundation for optimizers, since then there have been conceptual evolutions for gradient descent. In this tutorial we will be looking at the following with a coding example:

- Stochastic Gradient Descent (SGD)
- SGD with Momentum
- Adaptive Gradients (Adagrad)
- Root Mean Square Propagation (RMSprop) – further development of Adagrad

- Adaptive Moment Estimation (Adam) – Combination of SGD, momentum and RMSprop

4.1 Setting Up the Model and Data for Interactive Coding!

In this tutorial, we will build a regression model using various optimizers on the Scikit-learn California Housing dataset, which contains 20,640 samples with 8 features and their corresponding median house values. The data will be split into training and test sets, allowing us to evaluate the model's error and validation loss for each optimizer.

To compare the optimizers, we will use the same model architecture for each. The network is a simple sequential model with 8 input features, one hidden layer of 30 neurons, and a single output layer (Figure 4). Each optimizer will be applied to this model and trained for 30 epochs to evaluate its performance.

Follow the steps below to code along!

Step 1 – Import libraries

```
[1]
✓ 7s
#Importing packages
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import tensorflow as tf
from tensorflow import keras

import pandas as pd
import matplotlib.pyplot as plt

import numpy as np
import random

import seaborn as sns

#setting a seed value allows code reproducibility!
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)
```

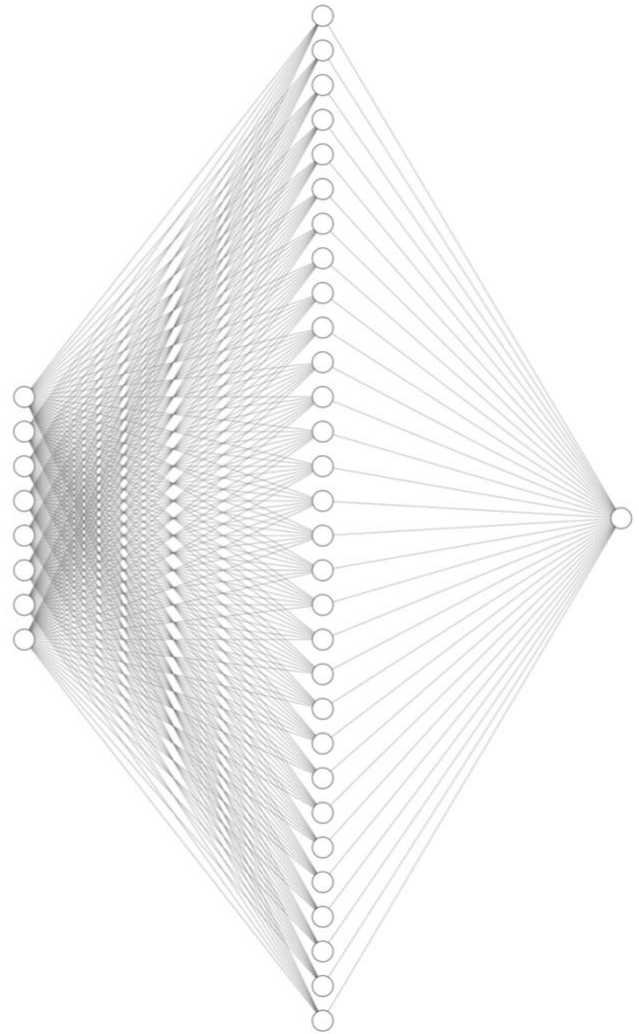


Figure 4 -Sequential Model Architecture for coding tutorial

Step 2 – Load and split the dataset into train, test and validation sets

```
[2]
#Loading dataset

housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target)
X_train, X_valid, y_train, y_valid = train_test_split(X_train_full, y_train_full)
```

Step 3 – Scaling the data as the 8 attributes of the datasets have various ranges

```
[3]
✓ 0s
#Scaling values

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_valid = scaler.transform(X_valid)
X_test = scaler.transform(X_test)
```

4.2 Stochastic Gradient Descent (SGD)

SGD is the same as gradient descent except instead of conducting gradient descent on every single sample, SGD takes random/stochastic mini batches of samples every iteration (Goodfellow.et.al, 2016). This randomness is why the seed is needed for reproducibility!

Step 4.A – Model with SGD optimizer

```
[4] ▶ #MLP with SGD Optimization

#Building the simple MLP
model_sgd = keras.models.Sequential([
    keras.layers.Dense(30, activation = 'relu',
                        input_shape = X_train.shape[1:]),
    keras.layers.Dense(1)
])

#Compiling the model
model_sgd.compile(loss = 'mean_squared_error',
                  optimizer = keras.optimizers.SGD()
                  )

#Applying the model on training dataset and validation testing
history_sgd = model_sgd.fit(X_train, y_train, epochs = 20,
                           validation_data = (X_valid, y_valid)
                           )

#Evaluating model on test dataset
mse_test_sgd = model_sgd.evaluate(X_test, y_test)

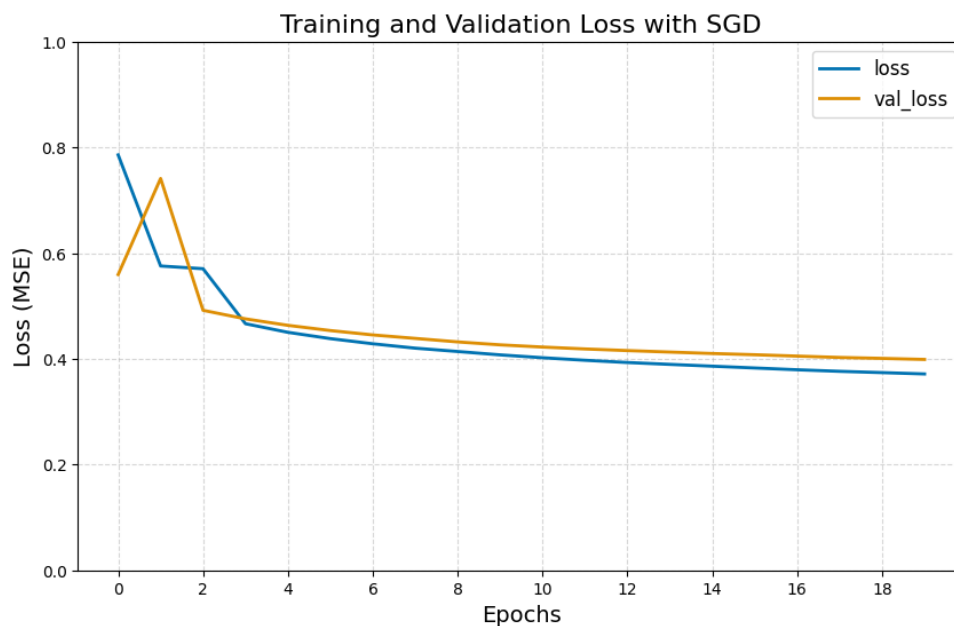
df = pd.DataFrame(history_sgd.history)

#Colorblind-friendly palette (2 colors for loss and val_loss)
palette = sns.color_palette("colorblind", n_colors=2)

# Plot with the palette
ax = df.plot(figsize=(10, 6), linewidth=2, color=palette)

ax.set_title('Training and Validation Loss with SGD', fontsize=16)
ax.set_xlabel('Epochs', fontsize=14)
ax.set_ylabel('Loss (MSE)', fontsize=14)
ax.set_xticks(np.arange(0, 20, 2))
ax.set_ylim(0,1)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(fontsize=12)
plt.show()
```

The code produces the following graph for the loss/error function for the training and validation loss:



4.3 SGD with Momentum

Imagine the gradient descent as taking a series of steps where we stop after each one. Due to the gradient equation not having a momentum term, it stops immediately when it gets to a minimum. We add momentum by adding a fraction of the past update(s)/steps(s) as we make each new update (Goodfellow.et.al, 2016). The momentum added gradient descent formula:

$$\Delta w(t) = -\eta \frac{\delta E}{\delta w} + \alpha \Delta w(t - 1)$$

Equation 6 - Gradient Descent with Momentum

α – Momentum coefficient:

- Carries the weight across the local minima
- Coefficient value ranges from 0.5 (moderate smoothing, cautious step) to 0.9 (strong momentum used in deep learning). Default value in keras is 0.9.

Adding momentum allows GD to skip over shallow local minima, if the local minima is too deep, momentum does not help. causes the GD to overshoot the global minimum slightly, however it prevents it from being stuck at a local minimum. Furthermore, it steps across plateaus faster.

Step 4.B – Model with SGD + Momentum = 0.9

```
[5] #MLP with SGD+Momentum Optimization

model_sgdM = keras.models.Sequential([
    keras.layers.Dense(30, activation = 'relu',
                        input_shape = X_train.shape[1:]),
    keras.layers.Dense(1)
])

model_sgdM.compile(loss = 'mean_squared_error',
                   optimizer = keras.optimizers.SGD(learning_rate = 0.001,
                                                       momentum = 0.9)
                   )

history_sgdM = model_sgdM.fit(X_train, y_train, epochs = 20,
                              validation_data = (X_valid, y_valid)
                              )

mse_test_sgdM = model_sgdM.evaluate(X_test, y_test)

df = pd.DataFrame(history_sgdM.history)

palette = sns.color_palette("colorblind", n_colors=2)
ax = df.plot(figsize=(10, 6), linewidth=2, color=palette)
ax.set_title('Training and Validation Loss with SGD + Momentum', fontsize=16)
ax.set_xlabel('Epochs', fontsize=14)
ax.set_ylabel('Loss (MSE)', fontsize=14)
ax.set_xticks(np.arange(0, 20, 2))
ax.set_ylim(0,1)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(fontsize=12)
plt.show()
```

The code should produce the following graph for the loss/error function for the training and validation loss:



4.4 Adaptive Gradients (Adagrad)

Until now, we have considered optimization approaches that apply the same learning rate to all parameters. Adaptive learning rate methods, like Adagrad, assign different learning rates to different parameters, adjusting them based on the past gradients (Goodfellow et al, 2016).

The Adagrad update rule is:

$$\Delta w(t) = -\frac{\eta}{\sqrt{v_t + \epsilon}} \frac{\delta E}{\delta w}$$

Equation 7 - Adaptive Gradient (Adagrad) formula

- $v_t = \sum_{r=1}^t g_r^2$ – Sum of the past squared gradients
- ϵ – Small constant added for numerical stability. This is added because early in the training the square root of the gradients is too small or even zero causing large weight updates, adding epsilon prevents this. Default value in keras: $\epsilon = 10^{-8}$.

This rule reduces the learning rate for parameters that have large gradients, allowing the optimizer to focus more on parameters that change slowly. Intuitively, if a weight has a large gradient, small changes in it can cause large changes in the loss, making aggressive updates unstable. Adagrad mitigates this by scaling the learning rate inversely with the accumulated squared gradients.

Step 4.C – Model with Adagrad with a modified learning rate of 0.01 (default 0.001 results in no model training)

```
[6] #MLP with Adagrad Optimization

model_Adagrad = keras.models.Sequential([
    keras.layers.Dense(30, activation='relu',
                        input_shape=X_train.shape[1:]),
    keras.layers.Dense(1)
])

model_Adagrad.compile(loss='mean_squared_error',
                      optimizer=keras.optimizers.Adagrad(learning_rate=0.01))

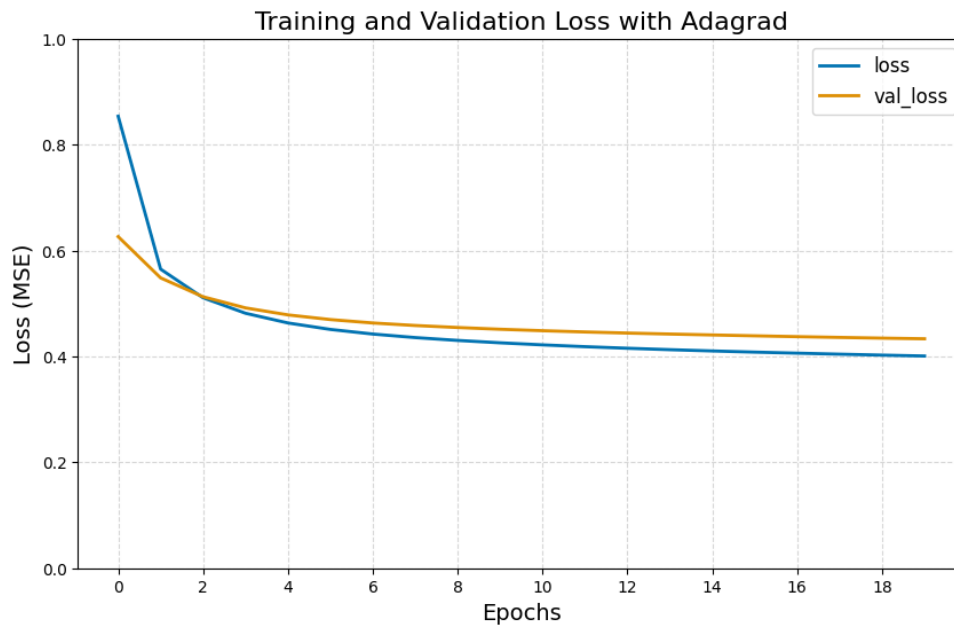
history_Adagrad = model_Adagrad.fit(X_train, y_train, epochs=20,
                                   validation_data=(X_valid, y_valid))

mse_test_Adagrad = model_Adagrad.evaluate(X_test, y_test)

df = pd.DataFrame(history_Adagrad.history)

palette = sns.color_palette("colorblind", n_colors=2)
ax = df.plot(figsize=(10, 6), linewidth=2, color=palette)
ax.set_title('Training and Validation Loss with Adagrad', fontsize=16)
ax.set_xlabel('Epochs', fontsize=14)
ax.set_ylabel('Loss (MSE)', fontsize=14)
ax.set_xticks(np.arange(0, 20, 2))
ax.set_ylim(0, 1)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(fontsize=12)
plt.show()
```

The code should produce the following graph for the loss/error function for the training and validation loss:



4.5 RMSprop

RMSprop changes the way gradients are accumulated in order to improve training stability (Goodfellow et al., 2016). Instead of summing all past squared gradients as in Adagrad, RMSprop keeps an exponentially decaying moving average of squared gradients, giving more weight to recent gradients. The update equations are:

$$\Delta w(t) = -\frac{\eta}{\sqrt{v_t + \epsilon}} \frac{\delta E}{\delta w}$$

Equation 8 - Weight update calculation

$$v_t = \beta * v_{t-1} + (1 - \beta)g_t^2$$

Equation 9 - Moving average of squared gradients

- β – decaying rate, default value in keras: rho = 0.9
- g_t – current gradient

This allows RMSprop to adaptively adjust the learning rate for each parameter based on recent gradient history.

Step 4.D – Model with RMSprop

```
[ ] #MLP with RMSprop Optimization

model_RMSprop = keras.models.Sequential([
    keras.layers.Dense(30, activation = 'relu',
                        input_shape = X_train.shape[1:]),
    keras.layers.Dense(1)
])

model_RMSprop.compile(loss = 'mean_squared_error',
                      optimizer = 'RMSprop'
                      )

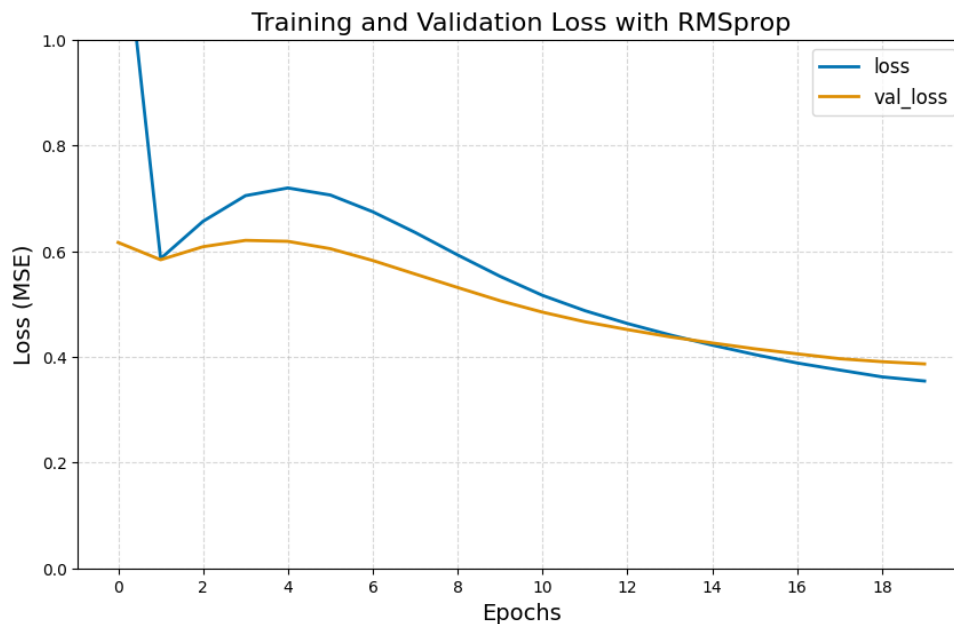
history_RMSprop = model_RMSprop.fit(X_train, y_train, epochs = 20,
                                   validation_data = (X_valid, y_valid)
                                   )

mse_test_RMSprop = model_RMSprop.evaluate(X_test, y_test)

df = pd.DataFrame(history_RMSprop.history)

palette = sns.color_palette("colorblind", n_colors=2)
ax = df.plot(figsize=(10, 6), linewidth=2, color=palette)
ax.set_title('Training and Validation Loss with RMSprop', fontsize=16)
ax.set_xlabel('Epochs', fontsize=14)
ax.set_ylabel('Loss (MSE)', fontsize=14)
ax.set_xticks(np.arange(0, 20, 2))
ax.set_ylim(0,1)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(fontsize=12)
plt.show()
```

The code should produce the following graph for the loss/error function for the training and validation loss:



4.6 Adaptive Moment Estimation (Adam)

Adam is an optimizer that combines stochastic sampling, momentum, and adaptive learning rates (Goodfellow et al., 2016). It estimates the first and second moments of the gradients as follows:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{\delta E}{\delta w}$$

Equation 10 - 1st Moment calculation

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \left(\frac{\delta E}{\delta w} \right)^2$$

Equation 11 - 2nd Moment calculation

The bias-corrected gradient estimates are then used to update the weights:

$$\Delta w = - \frac{\eta}{\sqrt{\frac{v_t}{1 - \beta_2} + \epsilon}} \left(\frac{m_t}{1 - \beta_1} \right)$$

Equation 12 - Bias Corrected Gradient Estimates for weights in Adam

This allows Adam to adaptively adjust the learning rate for each weight based on the estimated moments of the gradients.

Step 4.E – Model with Adam

```
[ ] #MLP with ADAM optimization

model_Adam = keras.models.Sequential([
    keras.layers.Dense(30, activation = 'relu',
                        input_shape = X_train.shape[1:]),
    keras.layers.Dense(1)
])

model_Adam.compile(loss = 'mean_squared_error',
                   optimizer = 'Adam'
)

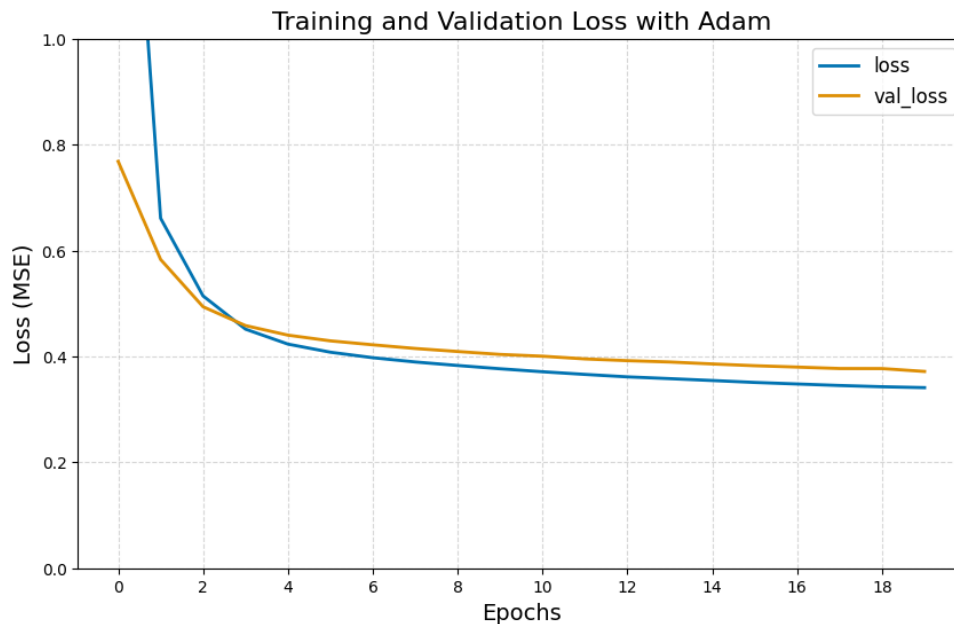
history_Adam = model_Adam.fit(X_train, y_train, epochs = 20,
                             validation_data = (X_valid, y_valid)
)

mse_test_Adam = model_Adam.evaluate(X_test, y_test)

df = pd.DataFrame(history_Adam.history)

palette = sns.color_palette("colorblind", n_colors=2)
ax = df.plot(figsize=(10, 6), linewidth=2, color=palette)
ax.set_title('Training and Validation Loss with Adam', fontsize=16)
ax.set_xlabel('Epochs', fontsize=14)
ax.set_ylabel('Loss (MSE)', fontsize=14)
ax.set_xticks(np.arange(0, 20, 2))
ax.set_ylim(0,1)
ax.grid(True, linestyle='--', alpha=0.5)
ax.legend(fontsize=12)
plt.show()
```

The code should produce the following graph for the loss/error function for the training and validation loss:



4.7 Comparison of Validation Loss across all optimizers

The training loss measures how well the model fits the training data. However, to evaluate whether the model generalizes to unseen data, we need to examine the validation loss. Unlike the training data, the validation data is not used to update the model weights during training, so it provides an unbiased estimate of the model's performance on new, unseen examples(Figure 5).

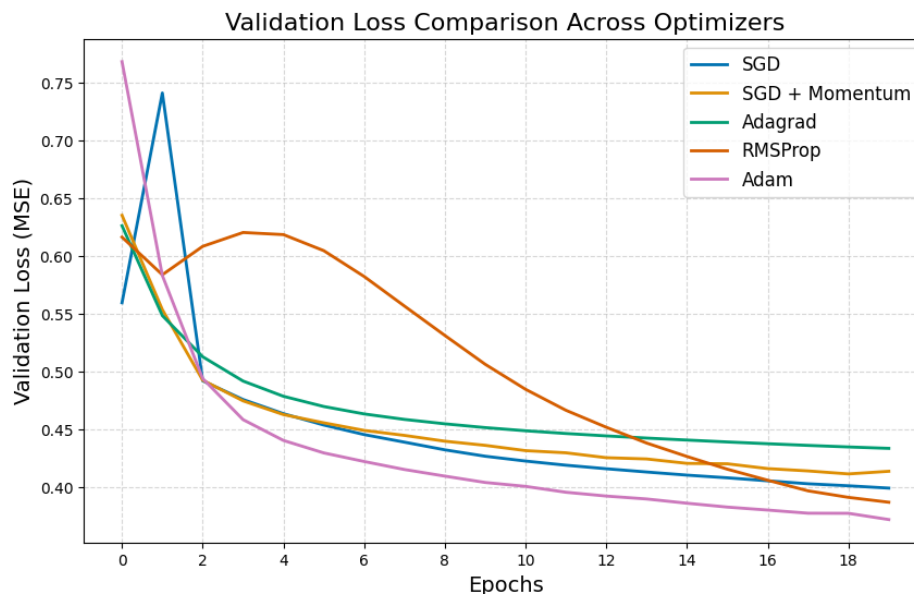


Figure 5 - Validation Loss Comparison across Optimizers

The results show that improving the optimizer progressively reduces the validation loss. Moving from SGD to SGD with momentum, then to RMSprop, and finally Adam, each step introduces techniques like momentum and adaptive learning rates that enhance convergence. Among these, Adam achieves the lowest validation loss, benefiting from its combination of stochastic updates, momentum, and adaptive learning rates.

In addition, we also measured their performance on the test set, which consists of data completely unseen by the model during training. The test MSE provides an unbiased estimate of how well each optimizer generalizes to new data.

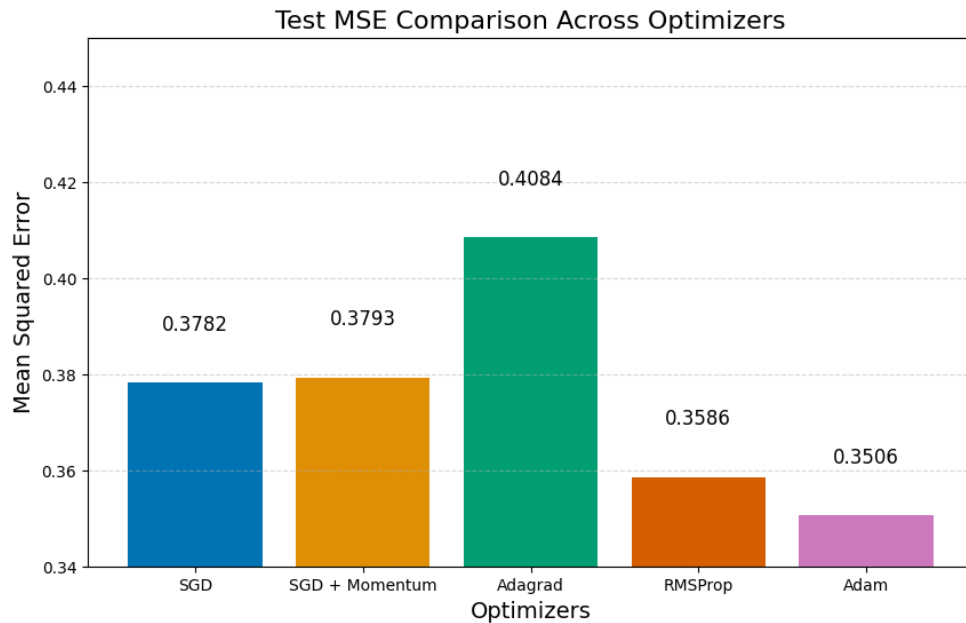


Figure 6 - Test Data MSE Comparison Across Optimizers

It is important to note that Adagrad exhibits the highest validation loss and test MSE among the optimizers. This is likely because Adagrad aggressively decreases the learning rate for parameters with large gradients, which can cause the model to slow down or get “stuck” early in training, especially when the network architecture or initial learning rate is not specifically tuned for this optimizer. As a result, it may fail to fully learn the underlying patterns in the data compared to more adaptive optimizers like RMSprop or Adam.

5 GitHub Repository Link

https://github.com/aa18afz/17074994_ML-NN_Tutorial_Assignment.git

6 References

CodeSignal, 2025. *Activation Functions: Introducing Non-Linearity with Sigmoid*. [Online]
Available at: <https://codesignal.com/learn/courses/neural-network-fundamentals-neurons-layers-and-the-forward-pass/lessons/activation-functions-introducing-non-linearity-with-sigmoid>

deeplearning.stanford.edu, 2025. *Multi-Layer Neural Network*. [Online]
Available at: <http://deeplearning.stanford.edu/tutorial/supervised/MultiLayerNeuralNetworks/>

geeksforgeeks, 2025. *ML | Common Loss Functions*. [Online]
Available at: <https://www.geeksforgeeks.org/machine-learning/ml-common-loss-functions/>

Goodfellow.et.al, 2016. Optimizing for Training Deep Models. In: *Deep Learning*. s.l.:MIT Press book, pp. 290-306.

IBM, 2021. *What is a neural network?*. [Online]

Available at: <https://www.ibm.com/think/topics/neural-networks>

Medium, 2024. *Understanding the Perceptron in Neural Networks*. [Online]

Available at: <https://medium.com/@noorfatimaafzalbutt/understanding-the-perceptron-in-neural-networks-7ddc6a2eca9d>

ScienceDirect, 2023. *Multilayer Perceptron*. [Online]

Available at: <https://www.sciencedirect.com/topics/computer-science/multilayer-perceptron>

towardsdatascience, 2021. *Neural Network Optimizers Made Simple: Core algorithms and why they are needed*. [Online]

Available at: <https://towardsdatascience.com/neural-network-optimizers-made-simple-core-algorithms-and-why-they-are-needed-7fd072cd2788/>